

Unit 2. Linux Commands and Utilities

2.1 Locating Commands, Internal & External Commands, Arguments, Options & Filenames, Online help

Internal & External Commands

The Linux system is command-based i.e things happen because of the commands that you key in. All UNIX commands are seldom more than four characters long.

They are grouped into two categories:

Internal Commands : Commands which are built into the shell. For all the shell built-in commands, execution of the same is fast in the sense that the shell doesn't have to search the given path for them in the PATH variable, and also no process needs to be spawned for executing it.

Examples: source, cd, fg, etc. time, kill, type, exit

How to get the list of Internal commands?

You can get only if you are in bash shell. Bash shell has a command called "help" which will list out all the built-in shell commands.

```
$ help
```

```
alias [-p] [name[=value] ... ]    bg [job_spec ...]
```

```
bind [-lpvsPVS] [-m keymap] [-f fi break [n]
```

```
builtin [shell-builtin [arg ...]] caller [EXPR]
```

```
case WORD in [PATTERN [| PATTERN]. cd [-L|-P] [dir]
```

```
command [-pVv] command [arg ...] compgen [-abcdefgjkuv] [-o option
```

```
.....
```

```
.....
```

External Commands : **External commands** are the **commands** that are executed by the **kernel**. Commands which aren't built into the shell. When an external command has to be executed, the shell looks for its path given in the PATH variable, and also a new process has to be spawned and the command gets executed. External Command existed in any **PATH** directory like */bin, /usr/bin* etc. like *ls, sed, mv*. These *commands* having independent existence in */bin, /usr/bin*, or any other directory. These commands will have a

process id running for it. For example, when you execute the “cat” command, which usually is at /usr/bin, the executable /usr/bin/cat gets executed.

Examples: ls, cat etc.

How to find out whether a command is internal or external?

In addition to this you can also find out about a particular command *i.e* whether it is internal or external with the help of **type** command :

\$type cat

cat is /bin/cat

//specifying that cat is external type//

\$type cd

cd is a shell builtin

//specifying that cd is internal type//

What is the difference between internal and external commands?

Internal commands are commands that are already loaded in the system. They can be executed any time and are independent. On the other hand, external commands are loaded when the user requests for them. Internal commands don't require a separate process to execute them. External commands will have an individual process. Internal commands are a part of the shell while external commands require a Path. If the files for the command are not present in the path, the external command won't execute.

What is the difference between internal and external commands?

- The commands that are directly executed by the shell are known as internal commands. No separate process is there to run these commands.
- The commands that are executed by the kernel are known as external commands. Each command has its unique process id.

Refer following link for more information

http://linuxcommand.org/lc3_lts0060.php

Syntax of UNIX/ Linux Command Lines

UNIX/ Linux command lines can be simple, one-word entries like the **date** command. They can also be more complex: you may need to type more than the command name.

A UNIX command may or may not have *arguments*. An argument can be an option or a filename. The general format for UNIX commands is:

command *option(s)* *filename(s)*

There isn't a single set of rules for writing UNIX commands and arguments, but you can use these general rules in most cases:

- Enter commands in lowercase.
- *Options* modify the way in which a command works. Options are often single letters prefixed with a dash (-) and set off by any number of spaces or tabs. Multiple options in one command line can be set off individually (like **-a -b**), or, in some cases, you can combine them after a single dash (like **-ab**).

Some commands, including those on Linux systems, also have options made from complete words or phrases, like **--delete** or **--confirm-delete**. When you enter a command, you can use this option style, the single-letter options (which all start with a single dash), or both.

- The argument *filename* is the name of a file that you want to use. If you don't enter a filename correctly, you may get the response "*filename*: no such file or directory" or "*filename*: cannot open."

Some commands, like **telnet** and **who** (shown earlier in this chapter), have arguments that aren't filenames.

- You must type spaces between commands, options, and filenames.
- Options come before filenames.
- Two or more commands can be written on the same command line, each separated by a semicolon (;). Commands entered this way are executed one after another by the shell.

UNIX has a lot of commands! Don't try to memorize all of them. In fact, you'll probably need to know just a few commands and their options. As time goes on, you'll learn these commands and the best way to use them for your job. We cover some useful UNIX commands in later chapters.

Let's look at a sample UNIX command. The **ls** command displays a list of files. It can be used with or without options and arguments. If you enter:

% ls

a list of filenames will be displayed on the screen. But if you enter:

% ls -l

there will be an entire line of information for each file. The **-l** option (a dash and a lowercase letter “l”) modifies the normal output of the **ls** command and lists files in the long format. You can also get information about a particular file by using its name as the second argument. For example, to find out about a file called *chap1*, enter:

% ls -l chap1

Many UNIX commands have more than one option. For instance, **ls** has the **-a** (*all*) option for listing hidden files. You can use multiple options in either of these ways:

% ls -a -l

% ls -al

You must type one space between the command name and the dash that introduces the options. If you enter **ls-al**, the shell will say “ls-al: command not found.”

help Command

help command displays information about shell built-in commands. Here's the syntax for it:

// syntax for help command

\$help [-dms] [pattern ...]

The pattern specified in the syntax above refers to the command about which you would like to know and if it is matched with any shell built-in command then **help** give details about it and if it is not matched then **help** prints the list of help topics for your convenience. And the d, m and s here are options that you can use with the **help** command.

// using help

\$help help

help: help [-dms] [pattern...]

Display information about builtin commands.

Displays brief summaries of builtin commands. If PATTERN IS specified, gives detailed help on all commands matching PATTERN, otherwise the list of help topics is printed.

Options:

- d output short description for each topic
- m display usage in pseudo-manpage format
- s output only a short usage synopsis for each topic matching
PATTERN

Arguments:

PATTERN Pattern specifying a help topic

Exit Status:

Returns success unless PATTERN is not found or an invalid option is given.

/* so that's what help command

does telling everything

about the command and

helping you out */

Options for help command

- **-d option** : It is used when you just want to get an overview about any shell built-in command *i.e* it only gives short description.
- **-m option** : It displays usage in pseudo-manpage format.
- **-s option** : It just displays only a short usage synopsis for each topic matching.

Using help with options

- **Using -d** : This option just lets you know about what a command does without giving you details about its options and other stuff.
- // using help with -d
-
- **\$help -d help**
- help - Display information about builtin commands.
- **Using -s** : This option is when you just want to know about the syntax of a command.
- // using help with -s
-
- **\$help -s help**

help: help [-dms] [pattern ...]

....

General Purpose Utilities

cal, date, who, whoami, tty, uname, passwd, echo, tput, be,script, wall, write, mail.

1. cal

It displays calendar of prev/current/next month if no argument is specified.

Displays calendar of specific year if specified.

\$ cal

Table 2.

Options	Function
-1	Display single month output.
-3	Display prev/current/next month output.
-s	Display Sunday as first day of week.
-m	Display Monday as first day of week.
-j	Display Julian dates.
-y	Display a calendar for current year.

2. date

Following command displays current date & time if no argument is given.

\$ date

Tue May 6 02:45:40 IST 2003

Formatted output for date command:

Field Descriptor	Function
%a	Abbreviated weekdays(SUN to SAT)
%A	Full weekday name
%b	Abbreviated month name
%B	Full month name
%c	Current date & time
%C	Century (00 to 99)
%d	Day of month 0 to 31
%D	Day as MM/DD/YY
%e	Day of month as decimal number
%h	Abbreviated month
%H	Hour 00 to 23
%I	01 to 12 (12 hour clock)
%j	Day of the year 001 to 366
%m	Month of the year 01 to 12
%M	Minute 00 to 59
%n	New time
%p	AM/PM
%r	Time in AM/PM notation
%T	Time in hh:mm:ss (23 hrs clock)

e.g. To display the date & time in the format

DATE=

TIME=

DAY=

Command is `$ date '+DATE=%D%n TIME=%T%n DAY=%A'`

To use multiple specifiers, enclose them within quotes and use + symbol before it.

`$ date +%m`

05

`$ date +'%h %d'`

May 03

3. **who**

This command displays the information of users who are currently logged in. Execute following command and write the output(only 3 users)

`$ who`

Execute the following options and write the use of it.

4. **who am i**

This command displays the information of the user who is currently logged in.

Execute the following command and write the output:

`$who am i`

5. **Stty (set tty)**

This command allows the user to set, view and change a wide variety of terminal settings.

E.g., end of line character, speed, end of file character, start, stop, suspend character etc.

Observe and the output of the following command:

`$stty`

`$stty-a`

To set terminal settings like erase character to ^e, observe observe the output of the following command

`$stty erase \^e`

6. **Uname**

Uname command is used to display basic information about the operating system and hardware. With options, Uname prints kernel details, and system architecture.

Uname is the short name for 'UNIX name'. Uname command works on all Linux and Unix like operating systems.

When you run Uname command without any options it just prints the core operating system name.

Syntax:

uname [OPTION]

- **-a option:** It prints all the system information in the following order:
Kernel name, network node hostname, kernel release date, kernel version, machine hardware name, hardware platform, operating system
- **-s option:** It prints the kernel name.
- **-n option:** It prints the hostname of the network node(current computer).
- **-r option:** It prints the kernel release date.
- **-v option:** It prints the version of the current kernel.
- **-m option:** It prints the machine hardware name.
- **-p option:** It prints the type of the processor.
- **-i option:** It prints the platform of the hardware.
- **-o option:** It prints the name of the operating system.

Refer link for more information

<https://www.geeksforgeeks.org/uname-command-in-linux-with-examples/>

7. passwd

This command changes the old password of the current user to the new one.

```
$ passwd
```

Changing password for the user pramod.

Changing password for pramod

(Current) UNIX password:

New password:

Retype new password:

8. echo

The following command without specifying its arguments within quotes squeezes tabs to a single space as below.

```
$ echo Hello      Dinesh
```

Hello Dinesh

To preserve spaces, include the arguments within quotes as given below:

```
$ echo "Hello      Dinesh"
```

Hello Dinesh

```
$
```

Note: By default, echo command does not interpret the escape characters like \n, \t when included in the text to display. You can enable their interpretation by -e option, and disable by -E option.

```
$ echo -e "hello\n"
```

```
Hello
```

```
$
```

As shown in above command new line has been inserted after the hello string.

9. tput

It initializes a terminal or query terminfo database

```
tput [-Ttype] capname [parms ... ]
```

```
tput [-Ttype] init
```

```
tput [-Ttype] reset
```

```
tput [-Ttype] longname
```

```
tput -S <<
```

```
tput -V
```

The **tput** utility uses the **terminfo** database to make the values of terminal-dependent capabilities and information available to the shell (see **sh(1)**), to initialize or reset the terminal, or return the long name of the requested terminal type. The result depends upon the capability's type:

Tag	Description
string	
	tput writes the string to the standard output. No trailing newline is supplied.
integer	
	tput writes the decimal value to the standard output, with a trailing newline.
boolean	
	tput simply sets the exit code (0 for TRUE if the terminal has the capability, 1 for FALSE if it does not), and writes nothing to the standard output.

Refer link for more information

https://www.tutorialspoint.com/unix_commands/tput.htm

10. Script

Script makes a typescript of everything printed on your terminal. It is useful for students who need a hardcopy record of an interactive session as proof of an assignment, as the typescript file can be printed out later with `lpr(1)`.

If the argument file is given, script saves all dialogue in file. If no file name is given, the typescript is saved in the file `typescript`.

Options:

Tag	Description
-a	Append the output to <i>file</i> or typescript, retaining the prior contents.
-c <i>COMMAND</i>	
	Run the <i>COMMAND</i> rather than an interactive shell. This makes it easy for a script to capture the output of a program that behaves differently when its stdout is not a tty.
-f	Flush output after each write. This is nice for telecooperation: One person does ‘ <code>mkfifo foo; script -f foo</code> ’ and another can supervise real-time what is being done using ‘ <code>cat foo</code> ’.
-q	Be quiet.
-t	Output timing data to standard error. This data contains two fields, separated by a space. The first field indicates how much time elapsed since the previous output. The second field indicates how many characters were output this time. This information can be used to replay typescripts with realistic typing and output delays.

11. wall command

`wall` command is used to send messages to all the users currently logged on to the system.

Example: If the superuser logged on `tty00` wishes to send messages to all users.

Syntax:

```
#wall
```

```
_____ # text message
```

```
_____
```

```
^d
```

```
#
```

Receivers of the message see the following on the screen

Broadcast messages from the root console on `tty00` Aug 1

```
_____ # text message
```

```
_____
```

```
<eof>
```

12. write command

The write command enables communication between users logged on to the system. The command sends a message to a specified user at the time the message is typed, provided the recipient user is logged on to system. Messages can be sent to and fro till the command is terminated using end of text character ^D on a line by itself. For two users to communicate to each other, each of them has to invoke write command.

Syntax:

```
$ write userid [terminal]
```

```
_____# text of message
```

```
_____
^d      # end of message
```

```
$
```

13. Talk command

The talk command can be used in the place of write for two way communication. The difference is that the viewers screen is divided into two halves where sender's screen is displayed as upper half and receiver's screen as lower half for both sender/receiver [chat facility on internet].

14. mail command

The mail command of UNIX/Linux facilitates the users to send messages to each other. The messages are stored in a mailbox in the /usr/mail directory of the system.

..

..

..

Syntax:**Sending mail**

```
$ mail userid [terminal]
Sub:
_____
_____} text of message
^d
$
```

Reading mail

```
$ mail [options]
{
  Most recent text message
}
&          # mail prompt
Commands at mail prompt
?          help summary
q          quit
x          exit without changing mail
p          print
d          delete
+          print previous
m user     forward mail to user
s [file]   save with header (default mbox)
w[file]    save without header
```

2.3 Navigating the File System Concepts: Files, Directories, Paths, Home Directory, Parent-Child Relationship, Handling Command.- pwd, cd, mkdir, rmdir,ls Ordinary Files --_, handling commands: cat, cp,rm,mv, - cmp,comm,diff

File Naming Conventions:

In Unix, filenames:

- Maybe up to 255 characters long (in Linux8.0) –Some characters may require special handling.
- Can contain digits, a dot(.), a hyphen(-), or the underscore symbol(_) anywhere.
- Can contain both upper case as well as lower case alphabets.
- Are case sensitive.
e.g., RESUME.TXT is not same as resume.txt.
file.txt is not as same as File.txt or File.Txt.
- Should not have a blank, or a tab.

By rule, Unix file names do not have to have ending extensions (such as .txt or .exe) as do some other operating systems.

However, certain applications with which you interact may require extensions, such as Adobe's Acrobat Reader (.pdf) or a web browser (.html). Thus the following are all valid Unix file names (note these may be any file type):

My_Stuff	a file named My_Stuff
my_stuff	a different file than above
mortgage.c	a C language program...can you guess what it does?
a.out	a C language binary executable
.profile	ksh default startup file

The Directory:

A directory is a file the solo job of which is to store the file names and the related information. All the files, whether ordinary, special, or directory, are contained in directories.

Unix uses a hierarchical structure for organizing files and directories. This structure is often referred to as a directory tree. The tree has a single root node, the slash character (/), and all other directories are contained below it.

Home Directory

The directory in which you find yourself when you first login is called your home directory.

You will be doing much of your work in your home directory and subdirectories that you'll be creating to organize your files.

You can go in your home directory anytime using the following command –

```
$cd ~
```

```
$
```

Here ~ indicates the home directory. Suppose you have to go in any other user's home directory, use the following command –

```
$cd ~username
```

```
$
```

The Home Directory:

The system places a user in his HOME directory upon login.

- Every user has a home directory, typically under/home or under/user
- Contains user- specific configuration files as well as user data.
- Represented by the ~ character.

Current Working Directory:

When you are in a shell, you are always in a directory on the system, and that directory is called the *current working directory*. When you first log in to the system, your home directory is the current working directory.

A container which stores information is known as File. You can treat it simply as a sequence of characters. It can be a document, picture, audio or video stream, data library, program file, application, or other collection of data.

Very important point here, Linux treats directories and devices as a file. In Linux system a directory is a folder which keeps filenames and other sub-directories. A Linux system considers all physical devices like your hard drive, CD-ROM, printer, memory etc. as files.

Imp: In Linux system a shell is also a file and kernel too.

Linux File Types

In UNIX there are different types of categories are there.

1. Ordinary File or Regular File
2. Directory File
3. Device File
4. Symbolic link file or soft link file
5. FIFO (Named Pipe)
6. Socket

1. Ordinary File

An ordinary file also known as regular file commonly is the most common file type in Linux system. All programs (C, C+, Java etc.) belongs to this file type.

Ordinary file is divided into two types such as Text File and Binary File.

Text File:: It contains only printable ASCII character within the range 0-127. Often we can view the content of a text file and can make sense out of it. Here every line ends with a new line character, created by ENTER Key.

For Example: All 'C', Source Codes, all Python Scripts, Shell Scripts

Binary File:: It contains both printable as well as non-printable characters within the ASCII range 0-255. We can not make any sense after viewing a binary file because it contains ASCII symbols.

For Example: All C and Java Object codes, all image, video, audio files are binaries. And also most Linux/UNIX commands are Binary File type.

When displaying the Binary File type with cat command it produces unreadable output and it may also disturb your terminal settings.

2. Directory File

Very important point you should keep in your mind that some users say and sometimes it is commonly said that a directory contains files and other directories. But it contains their names and inode number of a file or directory.

If you have directory and 10 files in a directory, there will be only 10 entries resides in the directory. It contains 2 attributes of a file.

- The File name
- Inode Number(A unique identification number of every newly created file which system assigns)

Unlike windows folders Linux directories never contain file data, file data are stored in data blocks.

Tip: *A directory contains filename and not the file's contents.*

3. Device File

Linux or UNIX considers every hardware devices as a file. They have different types of names also. All these device files are stored in the directory (/dev). A device file is indeed special it's not really a stream of characters. In fact it does not contain anything at all.

4. Symbolic link File

Symbolic link file also known as Soft link file. It is a very special type of file which contains path of the original file. Also you can say a symbolic link is a reference to another file.

5. FIFO (Named Pipe)

One of the main advantage and strength of Linux or Unix it has always been inter-process communication. Among the facilities provided by the OS are *pipes*, which connect the output of one process to the input of another. This is fine if both processes exist in the same parent process space, started by the same user, but there are circumstances where the communicating processes must use FIFOs, here referred to as *named pipes*. One such circumstance occurs when the processes must be executed under different user names and permissions.

Named pipes are special files that can exist anywhere in the file system. They can be created with the command `mkfifo`

6. Socket

A Unix/Linux socket is a special file which allows for advanced inter-process communication. A Unix Socket is used in a client-server application framework. In essence, it is a stream of data, very similar to network stream (and network sockets), but all the transactions are local to the filesystem. Unlike named pipes which allow only unidirectional data flow, sockets are fully duplex-capable.

7. Hidden Files

An invisible file is one, the first character of which is the dot or the period character (.). Unix programs (including the shell) use most of these files to store configuration information.

Some common examples of the hidden files include the files –

- `.profile` – The Bourne shell (`sh`) initialization script
- `.kshrc` – The Korn shell (`ksh`) initialization script
- `.cshrc` – The C shell (`csh`) initialization script
- `.rhosts` – The remote shell configuration file

To list the invisible files, specify the -a option to ls –

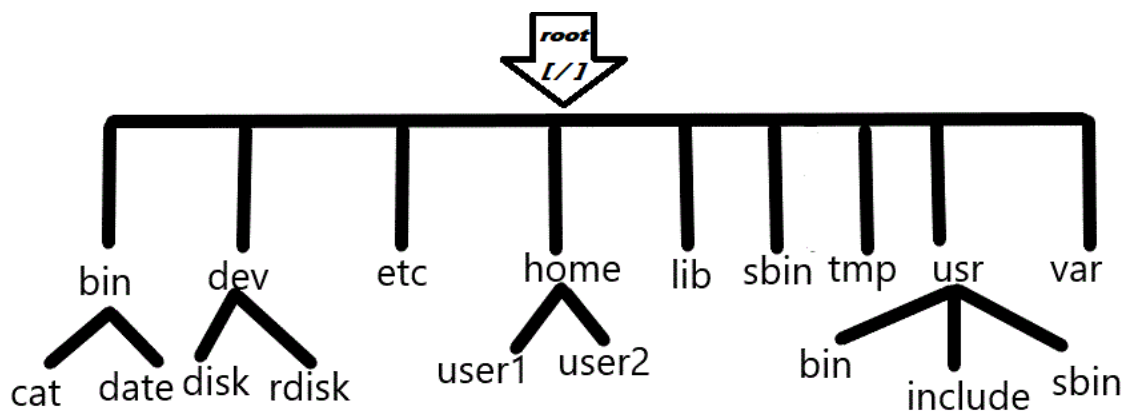
```
$ ls -a
```

```
.          .profile      docs      lib      test_results
..         .rhosts      hosts     pub      users
.emacs     bin          hw1       res.01   work
.exrc      ch07          hw2       res.02
.kshrc     ch07.bak      hw3       res.03
$
```

- Single dot (.) – This represents the current directory.
- Double dot (..) – This represents the parent directory.

Parent Child Relationship

All the files in Linux system are related to each other. The collection of all these files (Ordinary, directory, device etc.) is called as file system. And the filesystem in Linux is organized in a hierarchical structure. Directories/Files are here in a parent child relationship.



Linux file system is considered as single monolithic tree, an inverted one, root upwards. As you can see this tree is organized in hierarchical structure.

The directories and its contains.

Path	Type of Data Holds
/bin	Stands for binary. It contains command files of maximum commands
/dev	It contains all device files
/etc	This directory contains system configuration.
/home	Home directory contain all users
/lib	This directory contains all C language library files
/sbin	Super user bin. This dir contains command files of administrative commands
/stand/unix	this is unix kernel
/boot/vmlinuz	Path of Linux kernel path
/tmp	It contains temporary files, system wipes away this files in regular interval
/usr or /usr/bin	this directory contains command files of few additional commands
/usr/include	This dir. contain all C language header files
/var	This is variable part of the file system. Contains all user mailbox, crontab file of all user
/var/log	Contains system logs

Path of A file

A file path is the human-readable representation of a file or directory location on a computer system. The location of a file or a directory can be specified by either of two methods : by its *absolute* pathname or its *relative* pathname.

Absolute Pathnames:

- Absolute pathnames begin with a slash(/)
- It's a complete path to the file location
- It can be used any time you wish to specify a filename

Relative Pathname:

- Relative pathname do not begin with /.
- Specifies location relative to your current working directory.
- Can be used as a shorter way to specify filename.

Wildcard Characters:

Unix offers the facility to perform an operation on the set of files without having to specifying all the file names on the operation has to be performed. This is made possible by use of certain special characters in the command in place of the actual file names. These special characters are interpreted by Unix as a specific pattern of characters. Unix then compares all the filenames under the directory specified in the command to find out the filenames that match the pattern. The command is executed on the files with names that match the pattern.

Table1:

Character	Purpose
*	Matches none or one character or a string of more than one character
?	Matches exactly one character
[]	Matches exactly one of a specified set of characters

Refer following link for more information

http://dsl.org/cookbook/cookbook_8.html

https://homepages.uc.edu/~thomam/Intro_Unix_Text/File_System.html

Handling Command

1. pwd (print working directory)

This command displays the absolute path of the current working directory.

Execute the following command and write the output:

```
$pwd
```

2. cd (change directory)

The following command changes to/usr directory.

```
$cd/usr
```

Note: When *cd* command is executed without specifying directory name, then it will switch to the user's HOME directory.

```
$cd
```

3. mkdir(make directory)

If you are logged is as pramod:

The following command creates a subdirectory *dcn* under current directory. However , the new directory doesn't become the current directory.

```
$ mkdir dcn
```

Completes path names can also be specified with *mkdir* as:

```
$ mkdir/usr/pramod/dcn
```

4. rmdir(remove directory)

The following command removes the *dcn* subdirectory from the current directory.

```
$rmdir dcn
```

Note: A directory can be deleted only if it is:

- Empty(does not contains files or subdirectories)
- Not the current directory

Complete path names can also be specified with *mkdir* as shown in *mkdir command*.

5. ls (list)

Following command displays the file under current directory.

```
$ ls
abc.txt
dcn
$
```

Table1: Options for ls command:

Option	Functions
-a	Displays all files including.,...and hidden files
-A	Displays all but not. and ..
-C	List entries by column
-d	List directory entries instead of contents
-l	Use long listing format
-r	Sort in reverse order
-R	List subdirectories recursively

-s	Prints size of each file in blocks
-S	Sort by file size
-t	Sort by modification time,-u sort by access time
V	Sort by version
1	List one file per line
X	List in column format with row wise sorting
F	Classifies files, executables, directories
i	Shows icode no. of a file

Ordinary Files handling commands:

1. cat (concatenate)

It concatenates the contents of a specified file and also useful for creating and displaying contents of a file.

- To display the contents of a file *info.txt*,
\$ cat *info.txt*
- To create a file
\$ cat> abc.txt
Unix is a multi-programming operating system.
Unix is a robust and powerful operating system.
One should learn to use it.
<ctrl-d>
\$
Ctrl-d is used to save file and exit cat command.
Execute the following options and write the use of it.

2. cp(copy)

The following command copies *resume.txt* file to *newresume.txt*.

```
$cp resume.txt newresume.txt
```

The following command will copy all the files which start with *ch* and ends with exactly two character having extension txt from current directory to */usr/pramod/dcn* directory.

```
$ cp ch??txt /usr/pramod/dcn
```

Execute the following options and write the use of it.

3. mv (move)

The following command moves(renames) *resume.txt* to *biodata.txt*.

```
$ mv resume.txt biodata.txt
```

The following command moves *resume.txt* file to */usr/pramod* directory

```
$ mv resume.txt to /usr/pramod
```

The following command moves all the files with extension .sh from the current directory to */usr/pramod/dcn* directory

```
$ mv *.sh /usr/pramod/dcn
```

Option -i as available which behaves exactly like in cp.

4. The cmp command

The cmp command is used to compare files. When used without options, it uses two file names as arguments and displays the differences on the terminal.

Syntax: `cmp file1 file2`

The option `-l` gives a detailed list of the byte number and the differing byte in octal for each character that differs in both files.

5. The comm command

The comm command compares two sorted files. It compares each line of the first file with its corresponding line in the second file. It displays output in three columns: lines unique to the first file, lines unique to the second file, and lines common to both files, respectively.

Syntax: `comm file1 file2`

6. The diff command

To show differences between two text files, it tells you which lines in one file have to be changed to make the two files identical. Diff uses certain special symbols and instructions to indicate the changes that are required to make two files identical.

Syntax: `diff file1 file2`

2.4 File Attributes : File Permissions listing file, permissions, chmod Command

Various File access Permissions Commands:

The command *chmod* (short for charge made) allows to decide who can and can not read, write and execute a file.

To assign these types of permissions, the following three symbols are used.

Table 1. Commands and their syntax

Access type	Symbol	What it means
Read	r	Allows displaying, coping and compiling a file
Write	w	Allows editing and deleting of a file
Execute	x	Allows execution of a file

To specify then user to whom you are grating these type of permissions, use these three symbols.

Symbols	Description
u	The user or owner of file and directories
g	Members of the group o which user belongs
o	All other system users (users who are neither u or g)

Normally when a file is created, it is created with read, write and execute

Permissions to user and only read and execute permission for group members and other system users

Write permission is not given normally for these (group and other user) two categories. If it is required, the user can permissions by system commands.

HOW TO DETERMINE EXISTING PERMISSIONS:

The user can determine what permissions are currently in effect on a file or a directory by using command that produces a long listing of a directory's

Contents, that is **ls -l**

Example:

Assume the user is logged in under name 'vijay'

\$ ls -l <Enter>

The system displays the following.

Total 3

drwx - -x - -x vijay project 50 june 2 15:30 dir1

-rwx r - x r-x vijay project 1120 june 1 09:50 prog1

-rwx r - -r - - vijay project 2220 june 6 10:40 sample1

Permissions for the prog1 and sample1 files and directory dir1 are shown above

Permissions and their explanations for the above example are shown in the below table.

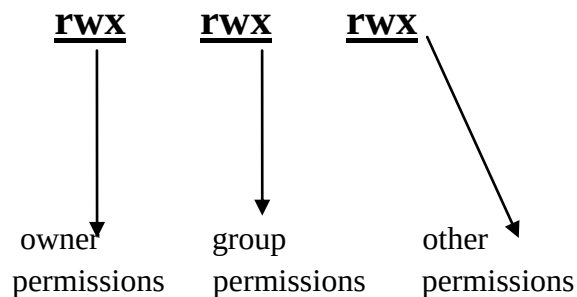
Permissions	Files/directory	Remarks
-rwx r - x r - x	Prog1 (file)	User/owner has got all the three permissions Group and Others have got only read and execute permissions
-rwx r - - r - -	Sample1 (file)	Sample1 defines write and execute permissions For group members and others
drwx r - - r - -	dir1 (directory)	dir1 directory denies permission for write and execute to group and others

Initial character, which describes the file or directory

‘-‘ (dash) symbolizes an ordinary file

‘d’ symbolizes a directory

There are nine possible permissions which can be given or refused in combination. They are written as a string of nine characters which are shown below:



e.g. rwx r-x r-x

In this, example user(owner) has read(r),write(w),execute(x) permissions and the members of group and other system users have read(r)and execute(x)

Permissions only.

HOW TO CHANGE EXISTING PERMISSIONS:

After you have determined what permission are to effect, you can change them by executing ‘*chmod*’ command in following format.

They are two ways of using ‘*chmod*’ command.

They are :

- 1) Symbolic method
- 2) Octal method

1) Symbolic Method

Syntax of symbolic method is as follows

`chmod who + permissions files(s) <Enter>`

where:

`chmod` - name of program (command)

who - one of three user groups (u,g,o or a)
 u = user/owner , g = group,
 o = others, a = all

‘+’ - means grant permissions
 ‘-’ - means deny permissions

permissions - any combination of three authorizations (r,w and x)
 files(s) - file or directory names(s) listed.

e.g. \$ chmod u+r prog1

In this, example, read permission has been added to the owner.

e.g. \$ chmod u-w <Enter>

In this, example owner has been denied write permission.

Note: If you omit scope (“u”, ”g” or “o”) the permission given apply to all the
 Three groups.

2) Octal Method:

This format requires to specify permissions using three octal numbers, ranging from 0 to 7. The octal code can be one to three digits long. The first, second, third digits set the “user”, “group” and “other” user permissions respectively. The integer used at each position dictates the permission for the file.

owner	group	other
r w x	r w x	r w x
(4) (2) (1)	(4) (2) (1)	(4) (2) (1)

The meaning of these integers is as follows

Integer	Description
0	is no permission
1	is execute permission
2	is write permission; 3- is write and execute permission
4	is read permission
5	is execute and read permission
6	is write and read permission
7	is read , write and execute permission

e.g. \$ chmod 750 sample1

(111 101 000) i.e. (r w x r- x - - -) for User (Owner), Group & Other member

This example changes access permissions for the file sample1 in the current working directory, the '7' gives permission for the owner of the file to read , write and execute file, the '5' gives permission for group members that file and '0' denies permission for users to read, write or execute the file "sample".

2.5 grep Family: Regular expressions, grep, egrep, fgrep, tr.

Various File Processing Commands:

The head command

You can use the head command to look at the beginning of a file.

Syntax:

head <option><filename>

e.g.

\$ head first.text

By default, you can only read the first ten lines of a file. You can change the number of lines to be displayed by specifying a number option, as shown in the following command:

Option:

Option	Description
n	Display specified number of lines

e.g.

\$ head -20 temp

In this command, you can view first 20 lines of a temp file

The tail command

The reverse of head is tail command, Using tail, you can view the last ten lines of a file.

Syntax

Tail option <filename>

e.g.\$ tail first.text

By default, you can read the only last ten lines of a file. You can change the number of lines to be displayed by specifying a number of option.

Options	Description
-n	Display specified number of lines
+n	Display all lines beginning from nth line

e.g.

\$: tail-20temp

In this command , you can view last 20 lines of temp file.

The tr command

The `tr` command is used to translate one set of character to another. This command can also

used to squeeze repeated character into one. Several commands have multi-column output and gap between columns is more than one space. In such cases, if user wants particular

column, `cut` cannot be used, since the column separator has to be used single character. Therefore,

the solution is to squeeze the several spaces between columns into a single space, and then use

`cut` command to extract the desired columns.

The `-s` option is used to specify squeezing of several occurrences of a character into One character

E.g. `$ who <enter>`

```
Param      tty2  june  1  9:06
```

```
Prashant   tty3  june  1 11:20
```

```
$
```

To make a column separator a single space, `tr -s` has to be used as shown

```
$ who > temporary <enter>
```

```
$ tr -s " " temporary <enter>
```

```
Param tty2 june 1 9:06
```

```
Prashant tty3 june1 11:20
```

Here the `-s` option of `tr` works on every record, squeezing , occurrences of string given In quotes(single space, in this case) to one space.

e.g . `$ echo abcdefaabbcc ! tr 'a-c' '1-3' <enter>`

```
123def112233
```

In this example, `echo` sends the letter `abcdefaabbcc` to `tr` command `tr` looks up each letter in its

first list if found, it translates it to the corresponding character in its second list. If the character is not found in first list, it is passed "as is" to the output

Another common usage of `tr` is case conversion.

```
$ tr "[a-z]" "[A-Z]" <enter>
```

```
CUSROW WADIA INSTITUTE OF TECHNOLOGY,PUNE
```

```
$
```

The above command converts all lowercase to uppercase.

The cut command

The cut command of unix/ linux is useful when specific columns from the specific file need to be extracted

Table for option of cut

Options	Function
-f	Display the fields specified
-c	Cutting of columns i.e. character b character
-d	Specifies the column delimiter

The -f options can be used to specify a field number if a tab character field separator.

e.g

```
$ cat data 1
```

```
Vishal 1979 2.10
```

```
Ladkat 1980 15.0
```

```
Param 1978 17.4
```

```
$ cut -f1, 3 data 1
```

```
Vishal 2.10
```

```
Param 17.4
```

```
$
```

In this example,-f1,3 tells cut out and keep the fields 1 and 3

e.g. \$ cat data2

```
param suresh jaldawar
```

```
$ cut -d `` -f1,3data2
```

```
Param jaldawar
```

```
$
```

The paste command

Paste command is used to paste several files together

Syntax

Paste [-s] [-d char]file1 file2....

\$cat f1 (file1 consider as f1)

a

b

c

d

```
$ cat f2 (file 2 consider as f2)
```

```
1  
2  
3  
4  
5  
6
```

```
$ paste f1 f2
```

```
a 1  
b 2  
c 3  
d 4  
 5  
 6
```

```
$
```

The join command

The join command enables you to merge data from a line in one file with data from corresponding

Line in second file on the basis of a key field that contains entries common to both of them.

The join command requires that both input files be stored according to the common field on which

They are joined.

Consider example

Here is file for day1 contain following data

```
$cat day1
```

```
ABC 2  
BCD 24  
DEF 6
```

```
$cat day2
```

```
ABC 2  
BCD 12  
DEF 22
```

```
$ join day1 day2
ABC  2  2
BCD  24 12
DEF  6  22
$
```

User can use other fields as the common field with the `-j` option.

e.g.

```
$join -j12 -j24 file1 file2
```

The split command

Split command is used to split file into smaller pieces. If file become too large, split lets user to

Handle the file in more manageable pieces.

Syntax: `split -number file tagname`

`-number` = specifies the number of lines to include in each file.

File = The file to split into smaller pieces

Tagname=by default, split builds the output creating the files named: xaa, xab, xac
And so on.

e.g.

```
$ split -500 studentlst newlist
```

The sort command

Many times you want to arrange contents of a file in specific order. Whether it is comparing files in alphabetic order or setting up database in numeric order Linux features a particularly useful tool for sorting files, named sort

Syntax

```
Sort <options> file1 file2 file3
```

Table for option of sort command

Command	Function
-tchr	Uses delimiter chart to identify fields
-n	Sorts numerically
-r	Reverses sort order
-c	Checks if file is sorted
-o fname	Places o/p in file fname

-u	Remove duplicates records
-m.n	Stops sort on nth column of (m+1)th field
+m.n	Starts sort after nth column of (n+1)th field
-k	Stop sort after kth field
+k	Start sort after skipping kth field
-m list	Merges sorted files in list

```
e.g.  $ cat tstfile
      Zebra
      Monkey
      Apple
      $
```

To ignore leading spaces in line when sorting use -b option

```
$ sort -b tstfile
Apple
Monkey
Zebra
$
```

```
e.g  $ sort numfile
12      Param
 2      Vishal
 5      Jaldwar
66      Suresh
$
$ sort -n numfile
2      vishal
5      jaldwar
12     param
66     suresh
$
```

In this example file is sorted on basis on of magnitude of number at start of line

The comm command

Comm command compares two sorted files. It compares each line of the first file with it's corresponding line in the second file. (Uniqueness & commonness)

It displays output in three columns,

lines unique to the first file,
lines unique to the second file and
lines common to both files, respectively.

It can be used to show any of the columns (Uniqueness & commonness) in the output. To drop a particular column, simply use its column number as an option prefix along with `-'.

Syntax: `comm file1 file2`

```
$cat file1
```

```
Diya
```

```
Prajakta
```

```
Adhirath
```

```
$cat file2
```

```
Vinayak
```

```
Prajakta
```

```
Abhishek
```

```
Akanksha
```

```
$comm file1 file2
```

```
Diya
```

```
        Vinayak
```

```
                Prajakta
```

```
        Abhishek
```

```
Adhirath
```

```
        Akanksha
```

```
$
```

The cmp command

The `cmp` command is used to compare files byte by byte when used without options, It uses two file names as arguments. It displays the first difference on the terminal. If two files are identical, `cmp` displays no message, but simply returns the prompt. (Difference)

Syntax: `cmp file11 file22`

The option -l(list) gives detailed list of

the byte no. and

the differing byte in overall for each character that differ in both the files.

```
$cat file11
```

This is Linux Operating System

```
$cat file22
```

This is Windows Operating System

```
$ cmp file11 file22
```

file11 file22 differ: byte 9, line 1

```
$
```

The diff command

To show differences between two text files also it tells you which lines in one file have to be changed to make the two files identical. Diff used certain special symbols and instructions to indicate the changes that are required to make two files identical.

a: add

c: change

d: delete

Syntax: diff file1 file2

```
$cat file1
```

Diya

Prajakta

Adhirath

```
$cat file2
```

Vinayak

Prajakta

Abhishek

Akanksha

```
$diff file1 file2
```

1c1

< Diya

```
---
>Vinayak
3c3,4
< Adhirath
---
Abhishek
Akanksha
```

The wc command

Linux features a universal word count program. It counts **no of lines, words and characters**, depending on the options used. It takes one or more files as it's arguments and displays a four column o/p.

e.g \$ wc infile
3 20 103 infile
Wc counts 3 lines, 20 words and 103 characters.

It offers 3 option to make a specific count.

Options	Function
-l	Counts only the number of lines
-w	Counts only the number of words
-c	Counts only the number of characters

The more command

If file is too large for it's contents to fit in one screen, it will scroll off your screen when you view it with cat. More allows the user to view a file, one screen at a time.

e.g. more chap01

You will see the contents of chap 1 on the screen , one page at a time.

Option	Function
Spacebar	Scrolls forward one screen
F	Scrolls forward one screen
B	Scrolls backward one screen
J	Scrolls forward one line
K	Scrolls backward one line
200G	Goes to line no. 200
G	Goes to last line of file

The pg command

This command is used to display large file, a screen at a time

Syntax

Pg filename <enter>

\$ pg data1

The above command will display the contents of data1, a screen at a time

The grep command

grep is the most powerful filters, it scans the file for the occurrence of a pattern, and can display the selected pattern.

Syntax: grep options pattern filename(s)

Grep requires expression to represent the pattern to be searched for, followed by one or more filenames.

Options	Function
-c	Displays count of number of occurrence
-l	Displays list of filenames
-n	Displays line numbers along with lines
-v	Displays all but lines matching expression
-i	Ignores case for matching
-h	Omits filenames when handling multiple files

The difference between grep, egrep and fgrep

One of the renowned search tool on Unix-like systems which can be used to search for anything whether it be a file, or a line or multiple lines in file is grep utility. It is very vast in functionality which can be attributed to the large number of options it supports like: searching using string pattern, or reg-ex pattern or perl based reg-ex etc.

Due its varying functionalities, it has many variants including **grep**, **egrep** (**Extended GREP**), **fgrep** (**Fixed GREP**), **pgrep** (**Process GREP**), **rgrep** (**Recursive GREP**) etc. But these variants have minor differences to original **grep** which has made them popular and to be used by various Linux programmers for specific tasks.

Main thing that remains to be investigated is what are the differences between the three main variants i.e. **'grep'**, **'egrep'** and **'fgrep'** of grep that makes Linux users choose one or the other version as per requirement.

Some Special Meta-Characters of grep

1. **+** – Equivalent to one or more occurrences of previous character.
2. **?** – This denotes almost 1 repetition of previous character. Like: **a?** Would match **'a'** or **'aa'**.
3. **(** – Start of alternation expression.
4. **)** – End of alternation expression.
5. **|** – Matching either of the expression separated by '|'. Like: **"(a|b)cde"** would match either **'abcde'** or **'bbcde'**.
6. **{** – This meta-character indicates start of range specifier.
Like: **"a{2}"** matches **"aa"** in file i.e. a 2 times.
7. **}** – This meta-character indicates end of range specifier.

The e in egrep stands for extended and the f in fgrep stands for fixed.

- egrep and fgrep are deprecated commands and should be avoided.
- egrep is equivalent to grep -E command and it allows the use of extended regex.
- fgrep is equivalent to grep -F command and it uses a fixed string for search and hence it performs a faster search.

Standard grep command

This is the standard version of grep designed for searching patterns using basic regular expressions. Using grep requires escaping meta-characters to generate the expected behavior.

Suppose I want to search for either tast or test. This is what I can use:

```
christopher@linux-handbook:~$ grep 't\(a|e\)st' test.txt
```

Extended Grep: Egrep or grep -E

Egrep was created to provide **extended** support for meta-characters. This means that it escapes these special characters by default. This saves time and generally makes the expressions easier to read.

The command I used above can be re-written as the much more legible:

```
christopher@linux-handbook:~$ egrep 't(a|e)st' test.txt
```

The `egrep` binary does not consistently support the `{` character. This can result in unexpected behaviors.

Some versions of `egrep` allow them to be interpreted properly as literal characters if escaped (`\{`). However, the best practice for matching a literal `{` is to enclose in square brackets `[{]`.

fgrep Command

fgrep or the **Fixed grep** or **grep -F** is yet another version of `grep` which is fast in searching when it comes to search for the entire string instead of regular expression as it doesn't recognize the regular expressions, neither any meta-characters. For searching any direct string, this is the version of `grep` which should be selected. `fgrep` searches for complete string and doesn't even recognize special characters as part of regular expression even if escaped or not escaped.

```
$ fgrep -C 0 '(f|g)ile' check_file
```

```
$ fgrep -C 0 '\(f|g\)ile' check_file
```

Like, when meta-characters were not escaped, `fgrep` searched for the complete string `"(f|g)ile"` in the file, and when the meta-characters were escaped, then the `fgrep` command searched for `"\ (f|g\) ile"` all characters as is in the file.